COMSM0122 Fundamentals of System Security

Part 1: The Trust Problem

Result: A, B, C and D all vote 100. Tally {100: 4}, quorum 3, committed value 100.

The consensus was achieved since the number of the matching votes (4) satisfied the quorum requirement floor(2n/3)+1 = 3 for n = 4. None of the nodes dictated the results because all nodes are treated equally by the simulator in terms of applying the same voting mechanism and any single one could be omitted without affecting the result. Quorum is what translates each node's vote into a collective decision and is selected in such a way that even if there was a lie, the rest of the votes would dictate the result. The proof of the consensus is the collection of the votes with signatures and the resulting counts, although in the current implementation, it is only printed but not stored by the simulator and cannot be proved to any third party (Task 8.5).

Part 2: Byzantine Behaviour

Result: D votes 50, the others vote 100. Tally {100: 3, 50: 1}, committed value 100.

D acts Byzantine since he voted for a number which was never put forward. Consensus is reached because the three honest votes are enough for reaching the quorum of 3, while D's one vote is too little to achieve this number. D was not recognized as a Byzantine node by the protocol. This protocol does not have to recognize Byzantine behavior if there are enough votes for reaching consensus, which is true if the number of honest nodes is larger than the threshold value. With n = 3f + 1, this four-node system can tolerate f = 1 Byzantine nodes. In case of two Byzantine nodes, they can prevent the system from reaching consensus (honest votes 2 < 3), or, together with the ability to be equivocal, split the votes.

Part 3: Equivocation

Result: D produces three messages: 100 to A, 50 to B and 75 to C, each with a different valid signature.

All messages are always signed using D's key, thus all receivers, independently verifying, see valid votes. This is why equivocation is more harmful than an invalid vote since the invalid vote is one lie told to everybody and just gets outnumbered, while equivocation makes every node see a different picture of the world, thus making different nodes come to different conclusions. An individual node cannot tell when it happens, since it always sees just one message. For detection, the messages seen by nodes have to be compared to each other, and two different signatures sent by the same sender in one round become undeniable, transferrable evidence of misbehavior (implemented in Task 8.1).

Part 4: Cryptographic Evidence and Key Theft

Result: the attacker uses secret-A to produce a vote from "A" for 999 with a valid signature.

The signature ties together nodeId: value with a secret, providing proof of origin and authenticity, as long as the secret is possessed by its rightful owner. Once the secret is obtained by an adversary, the forged message will match bit for bit with a message generated by A and will thus be verified successfully. In the protocol, we assume that the private keys are always known to their owners, and when this assumption breaks down, the adversary becomes equivalent to A and belongs to f. There are two additional flaws in the protocol simulation: keys can be predicted (secret-<id>) and the same shared secret is used both for verifying and generating signatures, which means that any verifier can also forge the messages.

Part 5: Cryptographic Evidence and Key Theft

Result: the validator set grows from A–D to A–D plus X1–X4 with no checks.

The adversary determines X1 through X4. Everyone is allowed to participate, and the simulator calculates quorum from the number of participants, such that with 8 nodes, quorum will be 6, while the adversary controls 4 out of 8 votes, much more than the tolerated f = 2. In part 7 (E3), this means no progress is possible when there are 4 Sybils and the adversary is able to commit its own value when there are 10. BFT only ensures that safety can be achieved as long as no more than f of the validators are faulty, but does not say anything about the faulty validators. Determining who the participants are constitutes an admission control and governance question, separate from consensus.

Part 6: Quorum Reasoning and Safety

Result: with quorum fixed at 2, votes split 2/2 and the simulator reports a safety violation: both 100 and 50 reach quorum.

Weakened rule is violated since two quorums of size 2 in a 4-node network do not need to intersect: quorums {A, B} and {C, D} do not intersect, thus they make decisions independently. What is now violated is safety (agreement), i.e., more than one value can be committed during one round. Using the right quorum of 3, any two quorums have 3+3-4 = 2 nodes in common, and there being at most one faulty node, at least one of those two nodes is an honest one. Since an honest node cannot vote for two values, two contradicting quorums cannot exist. In the general case, the condition is $2q - n > f$, resulting in $q \geq 2f+1$ when $n = 3f+1$. Future quorums have to intersect the previous ones since any decision made by the future quorum will contain a member of the previous one.

Part 7: Breaking BFT

Five attacks against the original simulator (scenarios/breaking_bft.js) broke safety three ways and availability twice; the most serious is an implementation bug, not a protocol flaw.

#	Attack	Outcome	Assumption violated	Category	Safety	Availability
E1	Replace B, C, D votes with garbage-signed 999	Commits 999	Votes are authenticated	Implementation	Failed	OK
E2	Steal B and C keys; D Byzantine	CoPart 7: Breaking BFTmmits 999, every signature valid	At most f = 1 faulty; keys stay secret	Infrastructure (key management)	Failed	OK
E3a	Add 4 Sybils voting 999	Quorum rises to 6; no consensus	Fixed, admitted validator set	Governance	OK	Failed
E3b	Add 10 Sybils voting 999	Commits 999	Fixed, admitted validator set	Governance	Failed	OK
E4	D equivocates during partition {A, B} | {C}	Weak quorum: A/B commit 100, C commits 50. Correct quorum: C cannot commit	Quorums overlap	Protocol parameter	Failed (weak) / OK (correct)	Reduced

#	Attack	Outcome	Assumption violated	Category	Safety	Availability
E5	Drop all messages from C and D	No consensus although all nodes are honest	Network delivers messages	Infrastructure (DoS)	OK	Failed

• E1 is a real flaw: simulator.js counts vote.value but never verifies(). Votes in quorum_tampering.js have no signature and are counted regardless. Unverifiable cryptography offers no security.

• E2: protocol is fine yet it commits attacker's value; once more than if participants are compromised, all guarantees are rendered meaningless and all forgeries are cryptographically valid.

• E3: Sybil attacks violate either property depending on the quantity: enough Sybils to prevent quorum violate availability; enough Sybils to form quorum violate safety.

• E4: equivocation combined with partition leads to contradictory commitments only when quorums are disjoint. With a correct quorum, the worst case scenario is waiting for part of the network.

• E5: BFT protocols would rather stop working than make a wrong decision; thus, denial-of-service attacks will target availability.
All five flaws may appear in practice. The 2022 Ronin Bridge attack is a typical E2: attackers managed to get 5 out of 9 validators' private keys.

Part 8: Hardening

hardened.js adds a HardenedSimulator that blocks or detects every Part 2–7 attack; scenarios/hardened.js replays each one against it. Every incoming vote passes through, in order:

1. Admission check (approved validator?)
2. Revocation and blacklist check
3. Signature verification
4. Evidence log and equivocation check
5. One vote per validator per round
6. Quorum check

7. Commit certificate

8.1 Detect equivocation

- Attack: D sends 100, 50 and 75 to A, B and C (Part 3).
- Strengthening assumption: a validator speaks with one voice to all.
- Modification: an EquivocationDetector stores {round, nodeId, value, recipient, signature} about each verified message, imitating the exchange of information among all parties. Two identical values from the same sender in one round cause the response EQUIVOCATION DETECTED, print the signed messages as evidence, and add the sender to blacklist.
- Demonstration: D is detected and blacklisted; A, B and C agree on 100.
- Weaknesses: detects but doesn't prevent; assumes that messages are exchanged indeed (n2 extra overhead); compares only one round at a time. It deals only with verified messages, thus it's impossible to frame an honest participant.

8.2 Strengthen key protection

- Attack: forging of a vote using stolen secret-A (Part 4).
- Assumption made stronger: a stolen key is valuable for a finite amount of time.
- Modification: a KeyRegistry with rotateKey(node) (randomly generated new key), revoke(id), and automatic rotation after N rounds.
- Demonstration: in the first round, the forgery succeeds, but A signs two different messages, making 8.1 suspect A. Upon rotation, the old key produces "bad signature". Revoke(B) prevents it from participating, leaving A, C, and D in the quorum. In case of key rotation after every 2 rounds, the key that was stolen in round 1 becomes invalid in round 3.
- Drawbacks: the protocol cannot distinguish between "A has lied" and "key has been stolen from A", making an honest node responsible. Rotation does not prevent stealing the new key by the attacker within the machine. The registry is a trusted central entity.

8.3 Prevent Sybil attacks

- Attackers: X1-X10 come together and cast 999 vote.
- Assumption enhanced: only those validators that have been approved can cast votes.
- Changes: a list of approvedValidators, any unknown ID would be considered out of question. The quorum will be calculated based on approved validators and not number of participating nodes, thus Sybils will not be able to increase quorum.
- Demonstration: all X cast votes are denied, quorum remains 3, 100 votes committed.
- Drawbacks: shifts trust to whoever controls the list. Cannot be used in permissionless environment.

8.4 Protect quorum rules

- Vulnerability: size() gives back 2 (Part 6).
- Stronger assumption: quorum parameters are safe.

- Modification: validateQuorum throws an exception if size is less than floor(2n/3) + 1, each time the simulator is started. Signature validations refuse unsigned votes used in the quorum_tampering.js exploit.
- Demonstration: "Unsafe quorum size: 2 < 3 for 4 validators"; the simulator will not run.
- Limitations: the developer who has access to validateQuorum can delete the function, hence code review and proper deployment remain important. The lower bound is verified but an oversize quorum would reduce availability.

8.5 Commit certificate

- Shortcoming: the original only prints COMMITTED: 100; nothing is done to prove that.
- Assumption made stronger: decisions can be audited retrospectively.
- Modification: every rejected proposal along with its {round, value, quorum, signatures} is recorded in the log of each commit. The method verifyCertificate() enables an auditor to verify the signatures and number of signers independently.
- Demonstration: valid certificate -> true; value changed to 999 -> false; only 2 signatures -> false.
- Constraints: the certificate is as good as the keys used to create it; hence the old keys should be kept for old certificates after key rotation.

Reflection

- Prevented completely: sybil nodes in a permissioned set (8.3), incorrect quorum configuration (8.4), unauthenticated or insufficiently authenticated voting messages.
- Detected/prevented partly: equivocation (8.1), key theft (8.2). The attacker will get an opportunity to take part in one round of the protocol and blame the wrong node for it.
- Complexity costs: evidence dissemination to all parties, keys management, controlled validator list. Every new solution demands a new trusted component such as a registry, validator list and protocol itself.
- Further conditions for production use: asymmetric signing scheme and HSMs, view change and leader change to guarantee liveness, DoS prevention and rate limiting, tamper-proof logging, elimination of equivocators.